
```
clear; close all; clc;
```

Section 1 - Dictionaries and Commands

```
% Airport Data (latitude, longitude) - also referred to as stored locations
```

```
airports = dictionary( ...  
'PHX',          struct('lat', 33.4352, 'lon', -112.3102), ...  
'LAX',          struct('lat', 34.0522, 'lon', -118.4085), ...  
'JFK',          struct('lat', 40.6413, 'lon', -73.7781), ...  
'MIA',          struct('lat', 25.7959, 'lon', -80.2870), ...  
'SEA',          struct('lat', 47.4502, 'lon', -122.3088) ...  
);
```

```
%Planes (Cruising Speed (knots), Fuel EFF(gal/hr))
```

```
planes = dictionary( ...  
'Boeing_737_800', struct('speed', 460, 'EFF', 850), ...  
'Airbus_A320',    struct('speed', 470, 'EFF', 825), ...  
'Boeing_777',     struct('speed', 490, 'EFF', 1800), ...  
'Gulfstream_G650', struct('speed', 516, 'EFF', 350), ...  
'Cessna_172',     struct('speed', 105, 'EFF', 8.5) ...  
);
```

```
%Pre-Defined Commands
```

```
%Function 1 - Asks user which airport, based on preestablished locations,  
%they want their destination to be from Phoenix Sky Harbor
```

```
function go = askd()  
while true  
    go = input('Where are you flying to from Phoenix? (LAX, JFK, MIA, or  
SEA) ', 's');  
    if ismember(go, {'LAX', 'JFK', 'MIA', 'SEA'})  
        break;  
    else  
        disp('Invalid location - Please enter LAX, JFK, MIA, or SEA.')  
    end  
end  
end
```

```
%Function 2 - Used to find name of unique user position
```

```
function name = askn()  
    name = input('Please provide destination name: ', 's');  
end
```

```
%Function 3 - Used to find latitude of unique user position
```

```
function coordlat = coordlat()  
    coordlat = input('Please provide destination latitude: ');  
end
```

```
%Function 4 - Used to find longitude of unique user position
```

```
function coordlon = coordlon()  
    coordlon = input('Please provide destination longitude: ');  
end
```

Section 3 - Calculations for Outputs and Outputs

```
%Finds change in longitude and latitude and converts to radians for
%haversine formula for distance given coordinates

dlat = difference(lat2, lat1);
dlon = difference(lon2, lon1);
distance = haversine(dlat, lat1, lat2, dlon);

%Calculating flight time and fuel consumption then storing them in new
%dictionary so that values are still sorted
list1 =
{'Boeing_737_800', 'Airbus_A320', 'Boeing_777', 'Gulfstream_G650', 'Cessna_172'};

%Blank lists and dictionary created so for loop can reference them
time_vals = {};
fuel_vals = {};
final_vals = dictionary();

for i = 1:length(list1)
    plane = planes(list1{i}); %picks plane from list1 based off index
    time = distance / plane.speed;
    fuel = time * plane.EFF;

    time_vals{i} = time; %adds new values to list for time values
    fuel_vals{i} = fuel; %adds new values to list for fuel values

    %Maps calculated values to relevant plane in the same way dictionaries
    %for locations and planes were mapped. Can reference the same way.

    %Example:
    % plane = final_vals(1) = 'Boeing_737_800'
    % plane.time will return how long it will take 'Boeing_737_800' to
    % reach designated destination based off calculations

    final_vals = insert(final_vals, list1{i}, ...
        struct('time', time_vals{i}, 'fuel', fuel_vals{i}));

    chosen = final_vals(list1{i});
    t = num2str(round(chosen.time, 2));
    f = num2str(round(chosen.fuel, 2));
    disp([list1{i}, ' will take ', t, ' hours to fly from PHX to ', go, ' and
burn ', f, ' gallons of fuel'])
end

list = cell2mat(list1);
time_numeric = cell2mat(time_vals);
fuel_numeric = cell2mat(fuel_vals);

%Plotting Flight Time vs Fuel Consumption
figure(1)
```

```

bar(time_numeric(1), fuel_numeric(1), 'r')
hold on
bar(time_numeric(2), fuel_numeric(2), 'g')
hold on
bar(time_numeric(3), fuel_numeric(3), 'b')
hold on
bar(time_numeric(4), fuel_numeric(4), 'c')
hold on
bar(time_numeric(5), fuel_numeric(5), 'y')
xlabel('Time (hours)')
ylabel('Fuel Consumption (gallons)')
title('Flight Time vs Fuel Consumption')
legend('Boeing737-800', 'Airbus-A320', 'Boeing777', 'Gulfstream-
G650', 'Cessna-172');
grid on

%Plotting Max # of People on Board vs CO2 Emissions per Person
polution = fuel_numeric .* 22.1;
figure(2)
scatter(189, polution(1)/189, 'r', 'filled')
hold on
scatter(180, polution(2)/180, 'g', 'filled')
hold on
scatter(396, polution(3)/396, 'b', 'filled')
hold on
scatter(18, polution(4)/18, 'c', 'filled')
hold on
scatter(4, polution(5)/4, 'y', 'filled')
xlabel('Max # of People on Board')
ylabel('CO2 Emissions per Person(lbs)')
title('Max # of People on Board vs CO2 Emissions per Person')
legend('Boeing737-800', 'Airbus-A320', 'Boeing777', 'Gulfstream-
G650', 'Cessna-172');
grid on

```

Published with MATLAB® R2025b